

Gestor de Comissões — Hardening de Segurança

Firebase (Fase 1)

Pendências de rollout em produção · gerado em 24/07/2026

1. O que já está pronto

- As **9 tarefas** do plano de hardening foram implementadas, cada uma revisada por subagente, e passaram por uma **revisão final do branch inteiro** (aprovada: “Ready to merge: Yes”, 0 Critical, 0 Important).
- npm test = 19/19 verde** (as 15 falhas históricas eram testes órfãos do Electron, removidos).
- Trabalho **mesclado no main** (fast-forward, commit 1ee2134) e **enviado para o origin/main** no GitHub. O código está publicado no repositório.

Entregue no código: criptografia AES-GCM dos dados de vendedor em `saved_reports`; regras Firestore com schema em todas as coleções; validador de `corretoras_config`; remoção do `bypass` de admin sem login (verificado no navegador); confirmação por e-mail + auditoria ao promover admin; suíte de testes de regras para o emulador; e limpeza do legado Electron.

2. O que ficou pendente (exige autenticação — sua ação)

Nenhum dos itens abaixo pôde ser executado nesta sessão porque o **Firebase não está autenticado** na máquina (`firebase login` não foi feito), e a autenticação/entrada de credenciais é uma ação sua, não do assistente. Além disso, há uma **ordem obrigatória** de rollout — descrita na seção 4 — sem a qual o app ao vivo quebra.

#	Pendência	Por que exige você
1	Deploy do frontend novo (hosting)	Pré-requisito da ordem correta; precisa de firebase login
2	Gerar a chave de equipe no app	Ação única de administrador, dentro do app em produção
3	Deploy das regras Firestore	Muda controle de acesso em produção; precisa de firebase login
4	Purga/regravação dos dados legados	Escrita/remoção de dados em produção; decisão e credenciais suas

3. Alerta crítico de ordem (evita queda de produção)

NÃO faça o deploy das regras antes de deploy do frontend novo.

As regras novas exigem o formato cifrado em `saved_reports`. O site ao vivo (`comissoesdp.web.app`) ainda roda o **frontend antigo**, que grava o formato antigo (com `summary/vendedoresDetalhes` em texto puro). Se as regras novas subirem antes do frontend novo, **tudo salvamento de relatório em produção passa a ser rejeitado** — os usuários veriam erro ao salvar. Por isso a ordem da seção 4 é obrigatória.

4. Sequência correta de rollout (passo a passo)

Pré-requisito: autenticar no terminal (uma vez):

```
firebase login
```

Passo 1 — Publicar o frontend novo. Depois disto, o app ao vivo passa a gravar o formato cifrado; as regras antigas ainda aceitam (não quebra nada).

```
npm run build
npx firebase-tools deploy --only hosting --project comissoesdp
```

Passo 2 — Gerar a chave de equipe (uma única vez). Entre no app como administrador → menu **Configurar corretoras** → botão **“Gerar chave de equipe”**. Sem essa chave, novos relatórios ficam salvos só localmente e não sincronizam com a nuvem.

Passo 3 — Publicar as regras Firestore. Só agora, com o frontend novo no ar e a chave criada, as escritas conformam ao schema exigido.

```
npx firebase-tools deploy --only firestore:rules --project comissoesdp
```

Passo 4 — Tratar os dados legados. Ver seção 5.

5. Dados legados em texto puro (o que fazer)

As regras novas só validam **escritas**. Documentos em `saved_reports` gravados **antes** deste trabalho ainda contêm nome+comissão de vendedor em texto puro e continuam legíveis por qualquer usuário aprovado. Identifique-os pelo campo `summary` presente (ou pela ausência de `encryptedSellerData`). Três caminhos:

- **Remover (mais simples):** pelo console do Firebase (Firestore → coleção `saved_reports`), apagar os documentos antigos que têm `summary`. Perde-se o histórico de ranking desses meses. **Ação destrutiva** — faça um export/backup antes.
- **Regravar (preserva histórico):** reprocessar os meses antigos pelo app novo e salvar de novo — o salvamento novo já cifra. Exige ter as planilhas de origem daqueles meses.
- **Aceitar (não recomendado):** deixar o histórico antigo como está; o critério “nada de PII em texto puro” passa a valer só para dados novos.

O assistente não executa remoção de dados em produção diretamente (ação destrutiva e irreversível). Se preferir, posso escrever um **script Node revisável** (usando `firebase-admin` com a sua conta de serviço) que você roda e audita — sem eu manusear suas credenciais.

6. Verificação após o rollout

- Salvar um relatório novo no app e conferir, no console do Firestore, que o documento em `saved_reports` tem só `encryptedSellerData` — nenhum `summary/vendedoresDetalhes` em texto puro.
- Confirmar que o Dashboard e a Analítica ainda mostram o ranking (prova de que a decifragem funciona).
- Rodar a suíte de regras num ambiente com Java: `npm run test:rules` (valida o comportamento real das regras contra o emulador).

7. Referências no repositório

- Plano completo:
`docs/superpowers/plans/2026-07-23-hardening-seguranca-firebase.md`
 - Spec de design:
`docs/superpowers/specs/2026-07-22-hardening-seguranca-firebase-design.md`
 - Regras: `firestore.rules` · Suíte do emulador:
`tests/cloud/firestore-rules-emulator.cjs`
 - Notas de retomada: `CLAUDE.md` (raiz do projeto).
-

Follow-ups de CI (não bloqueiam o rollout): rodar `test:rules` com Java; ampliar a suíte do emulador (escrita de reports, teste positivo “admin PODE atualizar usuário”, rejeição por tamanho de `encryptedSellerData`).